DAW - WEB INTERFACE DESIGN

UNIT 3.
USE OF STYLES

03/11/2026

Professor:
Begoña Rodríguez Ferreras

begona.rodfer.1@educa.jcyl.es

CSS STYLE SHEETS

Objectives:

- Analyze the different options for presenting information in web documents.
- Assess the importance of defining and applying a style guide in web application development.
- Select colors and other design elements for on-screen display.
- Create and use design templates.
- Recognize the possibility of modifying HTML tags.
- Define styles directly.
- Apply global styles using external style sheets.
- Identify the different properties of each element.
- Create style classes.

CSS STYLE SHEETS (II)

4.1. Introduction to CSS
4.2. How to Use External Style Sheets
4.3. Media: Adapting to the Viewing Environment or Device
    4.3.1. @media
    4.3.2. @import
4.4. The CSS Box Model
4.5. Inheritable CSS Properties
4.6. CSS3
4.7. Bootstrap 4: Responsive and Attractive Design
    4.7.1. Containers
    4.7.2. Grid System: Columns in Bootstrap
    4.7.3. Typography
    4.7.4. Colors
4.8. YUI (Yahoo! User Interface)

INTRODUCTION TO CSS

When working with web pages written in HTML and XHTML, it is desirable for them to have a specific style and a well-defined appearance.

The most efficient way to achieve this is to separate the part where the content is placed, represented by files programmed in HTML, from the part of the project that contains aspects related to presentation.

This is where CSS (Cascading Style Sheets) comes in, a style sheet language responsible for controlling the appearance of websites.

The use of CSS style sheets provides numerous advantages: it improves the accessibility of the document, reduces the complexity of maintenance, whether it is carried out by the original developer or by third parties, and also facilitates the display of the website on different devices.

BROWSER ENGINES AND CSS SUPPORT

- Not all browsers implement CSS features at the same pace.
- The rendering engine is the component responsible for interpreting HTML, CSS, and JavaScript and displaying web content.
- Modern browsers offer a high level of compatibility with current CSS standards.
- Main Rendering Engines
  - Blink → Chrome, Edge, Opera
  - Gecko → Firefox
  - WebKit → Safari

INTERNAL CSS STYLES

- CSS styles can be added directly to an HTML page using the <style> tag.
- The <style> section is typically placed inside the <head> element.
- This approach is useful for learning or styling a single page.
- For larger websites, it is recommended to use external CSS files.

```
<html>
 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=iso-8859
  -1"/>
  <title>
    Ejemplo de estilos CSS en el propio documento
  </title>
  <style type="text/css">
    p {
      color: black; font-family: Verdana;
    }
  </style>
 </head>
 <body>
  <p>Un parrafo de texto.</p>
 </body>
</html>
```

INTERNAL CSS STYLES (II)

FUNDAMENTAL

The main disadvantage of not using CSS files and instead defining styles directly within the HTML code is that, if a change to the defined styles is required, all pages that include the style to be modified must be updated.

HOW TO USE EXTERNAL STYLE SHEETS

Nowadays, external style sheets are commonly used.

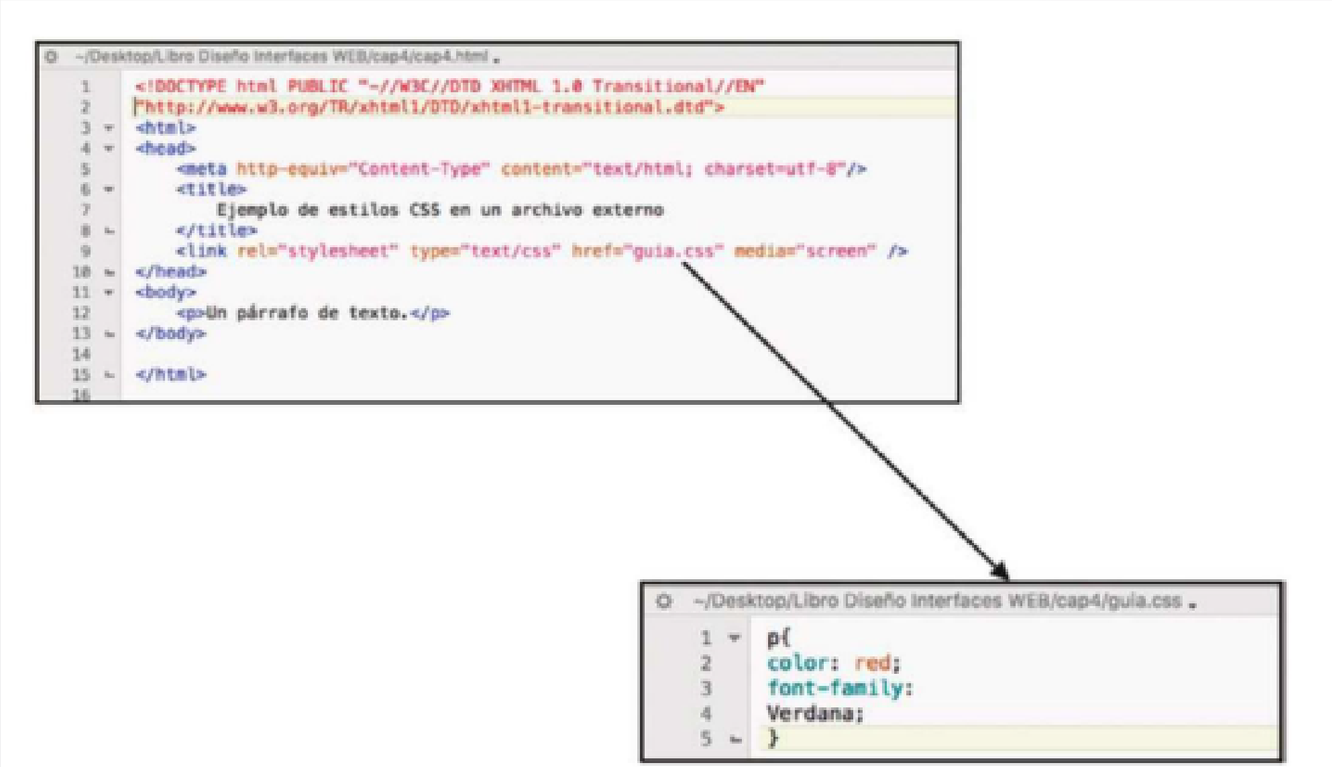
In this case, instead of having the code responsible for styling the website within the HTML document itself, that code is stored in a separate file.

From the main HTML page, the CSS file is accessed through the <link> tag.

The only particular aspect to keep in mind is that the file containing the styles must have the .css extension.

As many CSS files as needed can be linked to a web page

HOW TO USE EXTERNAL STYLE SHEETS (II)



HOW TO USE EXTERNAL STYLE SHEETS (III)

The purpose of each attribute used to link an HTML document to a CSS style sheet is as follows:

- **rel:** Specifies the type of relationship between the linked resource (the CSS file) and the HTML document. For CSS files, the value is stylesheet.
- **type:** Specifies the type of the linked resource. For CSS files, the value is text/css.
- **href:** Specifies the URL of the CSS file. This URL can be either relative or absolute and may point to an internal or external resource.
- **media:** Specifies the media type to which the CSS styles will be applied. CSS media types and their operation will be explained in more detail later.

HOW TO USE EXTERNAL STYLE SHEETS (IV)

Nowadays, the use of CSS in HTML is widespread because a single CSS file can be used to apply the same style guide to all the HTML pages of a website.

In this way, one of the most important principles of graphic design is achieved: **maintaining a consistent style across all the pages that make up the site**.

In addition, if a common style change is required across all pages, it is sufficient to modify the style sheet, and the changes will automatically be applied throughout the entire website linked to that CSS file.

HOW TO USE EXTERNAL STYLE SHEETS (V)

- Ejemplo:

```
<html>
 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
    <title>
            Ejemplo de estilos CSS en el propio documento
    </title>
    <link rel="stylesheet" type="text/css" media="screen, handheld"
    href="hojaEst.css">
 </head>
 <body>
  <p>Un párrafo de texto.</p>
 </body>
</html>
```

El código CSS (hojaEst.css):

```
p {
        color: red; font-family: Verdana;
}
```

MEDIA: ADAPTING TO THE VIEWING MEDIUM OR DEVICE

Style sheets make it possible t**o specify how a document is displayed depending on the type of viewing medium**.

These media refer to the channels through which a website is presented, such as audio, visual displays, printed documents, and others.

It is common practice to define a specific property for each type of medium.

To specify the target medium, several mechanisms can be used, either within the CSS style sheet or directly in the website's HTML file.

MEDIA: ADAPTING TO THE VIEWING MEDIUM OR DEVICE (II)

Media Type	Description	Current Status
all	All media types.	✅ Commonly used (default)
print	Printers and print preview.	✅ Commonly used
braille	Tactile devices using the Braille system.	❌ Deprecated
embossed	Braille printers.	❌ Deprecated
handheld	Handheld devices such as mobile phones and PDAs.	❌ Deprecated
projection	Projectors and presentation devices.	❌ Deprecated
screen	Computer, tablet, and smartphone screens.	✅ Most commonly used
speech	Speech synthesizers and screen readers.	✅ Supported
tv	Televisions and low-resolution display devices.	❌ Deprecated

@MEDIA

The @media rule specifies the target media types. Through this feature, a single style sheet can contain rules for different media. If the same style applies to multiple media, their names can be specified as a comma-separated list.

In modern web development, media queries are mostly used to create **responsive designs** based on screen characteristics rather than media types:

```
/* Mobile */
@media (max-width: 768px) {
    .menu {
        display: none;
    }
}
```

```
/* Dark mode */
@media (prefers-color-scheme: dark) {
    body {
        background: #222;
        color: white;
    }
}
```

```
/* Printing */
@media print {
    nav,
    footer {
        display: none;
    }
}
```

@IMPORT

Using @import, it is possible to import style rules from other CSS files and specify the media or media types to which the imported styles should apply. In this case, unlike @media, the URL of the CSS file containing the style sheet is specified first, followed by the target medium to which the styles will be applied. It is important to note that @import rules must appear before all other style rules.

If the styles from the imported file are to be applied to multiple media types simultaneously, the names of those media types should be separated by commas.

```
@import url("filename.css") mediaType;
```

```
@import "ficheroEstilo1.css" print;
@import "ficheroEstilo2.css" screen;
@import "ficheroEstilo3.css" print, screen;
@import "ficheroEstilo4.css"; (Por defecto se aplica a todo)
```

@IMPORT (II)

Although the @import rule is still supported by current browsers, external style sheets are typically linked using the <link> element in the HTML document. The @import rule is mainly used in specific cases, such as organizing CSS files within a larger stylesheet.

THE CSS BOX MODEL

The box model is one of the most important features of style sheets.

When an HTML page is created, the use of each tag simulates what could be considered a box, giving each one an identifier indicated by the tag name. This box system makes it possible, from the CSS file, to select each box and apply the desired style to it.

Using CSS style sheets allows the modification of all the characteristics of each of the boxes that have been created. The following section analyzes each of these parts through an example that incorporates new properties

THE CSS BOX MODEL (II)

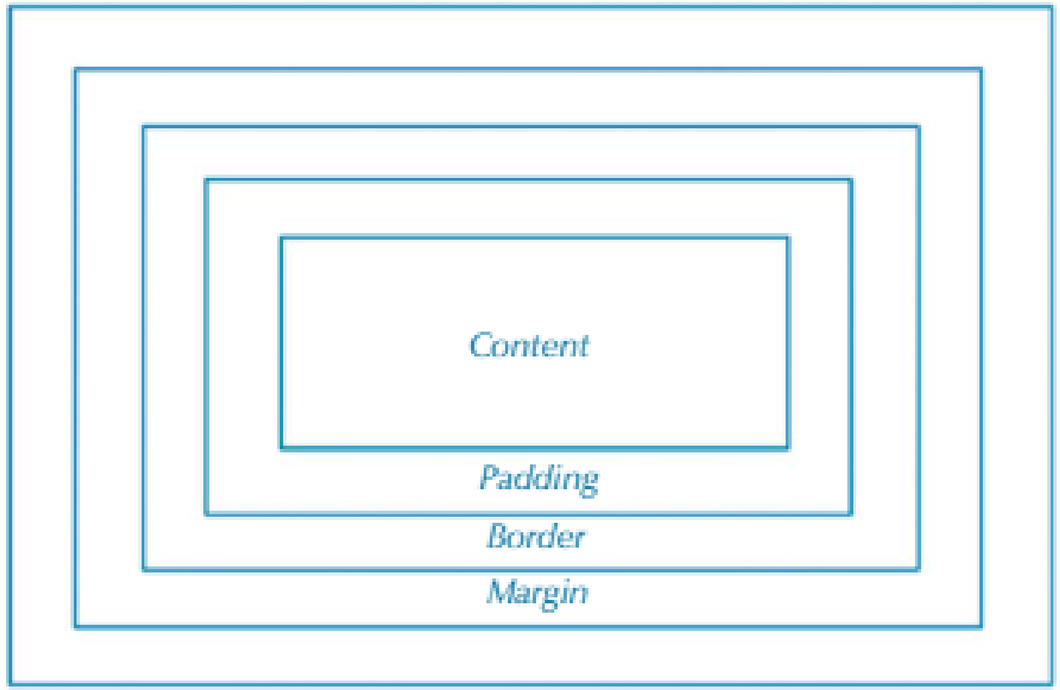


Figura 4.3
Diagrama modelo de cajas en CSS.

THE CSS BOX MODEL (III)

The main properties used in the **box model** are **width** and **height**, which are related to the dimensions of the content box. These define the width and height of the area where the box's content is displayed.

Another important property of the content area is the **background** of the box. A distinction is made between:

- **background-image** (background image): which refers to the image displayed behind the content and padding area.

- **background-color** (background color): which is the color shown behind the content and padding area.

THE CSS BOX MODEL (IV)

For example, if the following code is entered:

```
article {
    background: #5DEAAB;
    height: 100px;
    width: 200px;
}
```

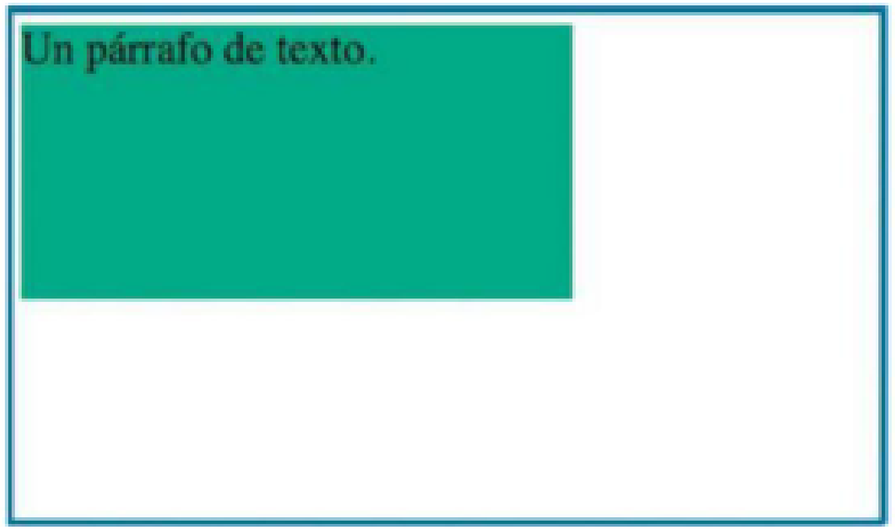


Figura 4.4
Color de fondo.

THE CSS BOX MODEL (V)

The second area is the padding area, known as **padding**.

It is the space that exists between the content box and the border.

The characteristics of this area that can be modified are its height and width.

Therefore, four possible zones can be distinguished: bottom area, top area, left area, and right area. The properties that define these zones are padding-top, padding-right, padding-left, and padding-bottom.

```
article {
    padding-top: 10px;
    padding-right: 30px;
    padding-bottom: 10px;
    padding-left: 20px;
}
```

THE CSS BOX MODEL (VI)

If only the padding property is specified, **the same value will be applied to all sides**. In the following example, the bottom, top, left, and right edges would all have a value of 10 px.

```
article {
    padding: 10px;
}
```

There are additional ways to use this rule that simplify its application. These are based on using **the padding property followed by several values separated by spaces.** If **two values** are provided, the first one is assigned to the top and bottom edges, while the second value is applied to the left and right edges. If **three values** are provided, the first and third values are applied to the top and bottom edges, respectively, while the second value specifies the size of the left and right edges.

THE CSS BOX MODEL (VII)

The next area that can be seen in the box model diagram is the **border**, known as border.

This area encloses both the content and the padding, that is, the content and padding areas.

It is possible to modify its **width**, **style** and **color**.

In addition, just as with padding, the same format can be applied to all four sides of the box at once using only the border property, or each side can be configured individually.

```
article {
    border: 1px solid #000000;
}
```

THE CSS BOX MODEL (VIII)

In cases where a different format is desired for each side, some of the individual properties intended for this purpose can be used:

Property	Description
border-top border-right border-bottom border-left	Modify the width, style, and color of each side of the border simultaneously.
border-width border-style border-color	Individually modify the width, style, and color of all four sides at once.
border-top-width	Modifies the width of the top border.
border-top-style	Modifies the style of the top border.
border-top-color	Modifies the color of the top border.

THE CSS BOX MODEL (IX)

Finally, the space that surrounds the other CSS elements is called the **margin**.

It separates and positions boxes within the layout.

The individual properties are **margin-top**, **margin-right**, **margin-bottom** and **margin-left**. Their behavior is the same as that of the padding property.

```
article {
    margin-top: 10px;
    margin-right: 10px;
    margin-bottom: 10px;
    margin-left: 10px;
}
```

NOTE

Margins have a peculiar behavior called **margin collapsing:** When two boxes touch, the distance between them is the value of the larger margin, rather than the sum of both margins.

INHERITABLE CSS PROPERTIES

Inheritance in CSS is the mechanism by which certain properties of a "parent" element are passed on to its "children." Not all CSS properties are inheritable; for example, margins are not inherited because it is unlikely that a child element needs the same margins as its parent. When the value of a CSS property is set on an element, its descendant elements automatically inherit that property's value.

```
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1"/>
    <title>Example of Style Inheritance</title>

    <style type="text/css">
        body { color: blue; }
    </style>
</head>

<body>
    <h1>Page Title</h1>
    <p>A not very long paragraph of text.</p>
</body>
</html>
```



INHERITABLE CSS PROPERTIES (II)

In the example, the body selector only sets the text color for the <body> element. This property is automatically inherited, which means that the descendant elements display that same color. However, although style inheritance is applied automatically, its effect can be overridden by explicitly assigning a different value to the inherited property.

There are properties whose values are not inherited, such as borders (border). That is, if a style defining a border around the body is applied, it will not be applied to the rest of the HTML elements.

Next table summarizes some of the properties that are inherited by default in HTML elements.

INHERITABLE CSS PROPERTIES (III)

Summary of Properties Inherited by Default in HTML:

Property	
color	text-align
font	visibility
letter-spacing	white-space
direction	border-collapse
word-spacing	quotes
line-height	list-style-image

INHERITABLE CSS PROPERTIES (IV)

As mentioned earlier, **there are certain properties that are not inherited from parent elements to child elements**.

However, **there is a mechanism that allows an element to inherit the value of a property.**

To do this, the values inherit, initial, and unset are used, which modify the property's value with respect to inheritance.

a) **inherit:** Applies the value of the property from the parent element.
b) **initial:** Applies the property's initial (default) value.
c) **unset:** Inherits the value of the property from the parent element; if no inherited value exists, the initial value is applied.

INHERITABLE CSS PROPERTIES (V)

In this example, the h1 element inherits all the properties defined for the body element. The text color is applied automatically because it is one of the properties that can be inherited, and since no different color value is specified for h1, it will appear in red. In addition, in this example the border properties will also be inherited because this is explicitly indicated by the inherit value.

```
body {
    border-width: 10px;
    border-color: blue;
    color: red;
}

h1 {
    border: inherit;
}
```

CSS3

CSS3 (Cascading Style Sheets 3) is the third version of the CSS style sheet language. This section presents some of the new features introduced compared to its previous version, CSS2.1.

This update includes all the features of its predecessor and, in addition, provides greater flexibility and dynamism in web page design.

As for the programming-related aspects, they remain unchanged: the same structure of properties and selectors used throughout this chapter is still employed.

Note: Although these features were originally introduced in CSS3, modern web standards no longer use version names such as CSS3 or CSS4. The language is now referred to simply as CSS.

A) MULTIPLE BACKGROUND IMAGES

In this new version, it is possible to apply several background images to a single element. In the following example, you can see how multiple images are used for the background and how the position of each image is specified after its URL.

```
#mainContent {
    width: 500px;
    height: 500px;
    background:
        url(image1.gif) bottom right no-repeat,
        url(image2.jpg) center center no-repeat,
        url(image3.jpg) repeat;
}
```

B) NEW PROPERTIES FOR ELEMENT BACKGROUNDS

CSS3 introduces new properties that define the characteristics of element backgrounds. These include background-origin, background-clip, and background-size.

- **background-origin** specifies the origin position of a background image.
- **background-clip** defines how a background image behaves in relation to an element's border, determining whether it should be displayed underneath the border or not. This property can be applied to all container boxes known as border-box, padding-box, and content-box.
- **background-size** allows you to define the size of background images.

C) ROUNDED CORNERS

This property allows the corners of elements to be rounded using the border-radius property. In modern browsers, the standard border-radius property is fully supported, so vendor-specific prefixes are no longer required.

```
div {
    border: 2px solid #ffffff;
    border-radius: 9px;
}
```

D) SHADOWS

This version introduces a property for creating shadows on elements, including text. The property used is box-shadow, which is composed of four values: horizontal offset, vertical offset, blur radius, and shadow color. These values always appear in the same order

```
div {
    box-shadow: 5px -9px 3px #000000;
}
```

E) COLOR TRANSPARENCY

Another feature introduced in CSS3 is the ability to add transparency to colors, ranging from a completely solid color to a fully transparent one. The property that defines transparency is called opacity, and it takes a decimal value between 0.0 and 1.0, where 0.0 represents maximum transparency and 1.0 represents maximum opacity.

```
div {
    opacity: 0.5;
}
```

Modern note: While opacity is still commonly used, modern CSS often applies transparency directly to colors using rgba() or hsla() values. This allows the background to be transparent without affecting the opacity of the element's content.

F) MULTI-COLUMN TEXT

CSS3 introduced the ability to lay out text in multiple columns using several properties: column-width (defines the width of the columns), column-gap (defines the space between columns), and column-rule (creates a line between columns).

When CSS3 was first introduced, support for these properties was experimental and required browser-specific prefixes such as -moz- for Firefox and -webkit- for Chrome and Safari. Today, these properties are fully standardized and supported by modern browsers without vendor prefixes.

```
div {
    column-width: 20em;
    column-gap: 10px;
    column-rule: 2px solid #ffffff;
}
```

```
div {
    column-count: 3;
    column-gap: 20px;
}
```

F) MULTI-COLUMN TEXT (II)

Multi-column layouts are still supported and occasionally used for articles, magazines, news sites, and long blocks of text.

However, modern web development more commonly relies on **Flexbox** and **CSS Grid** for page layouts, reserving multi-column properties mainly for text-flow formatting rather than overall page structure.

G) ANIMATIONS

One of the most important innovations introduced with CSS3 was the ability to create animations directly in CSS, eliminating the need for external technologies or browser plugins.

Today, CSS animations remain widely used and are a fundamental part of modern web development. They are commonly used for interface transitions, hover effects, loading indicators, menus, buttons, and visual feedback.

BOOTSTRAP. AGILE AND ATTRACTIVE DESIGN

Bootstrap, originally created by Twitter, is one of the most popular CSS frameworks in use today.

A framework is a collection of tools, styles, and reusable components that help developers build responsive and visually appealing websites more efficiently.

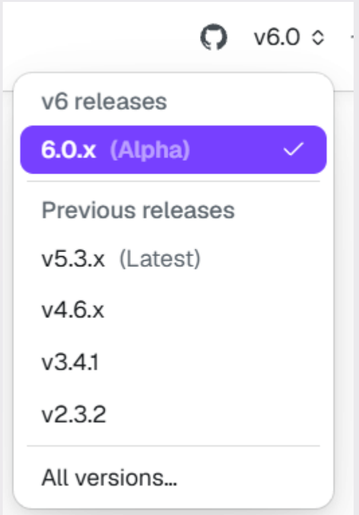
Bootstrap includes a wide range of resources such as typography, buttons, forms, navigation components, and a flexible grid system. The current major version is Bootstrap 5.

BOOTSTRAP (II)

Web Resources

Bootstrap is an **open-source framework** and can be accessed through its official website:

- https://getbootstrap.com/
- Latest stable release: https://getbootstrap.com/docs/5.3/getting-started/introduction/
- By default is going to bring to the newer Alpha version: https://getbootstrap.com/docs/6.0/
- https://startbootstrap.com/themes



BOOTSTRAP (III)

This tool allows developers to create a wide variety of user interfaces by combining the components and utilities provided by the framework. Bootstrap is compatible with all major modern web browsers, including Google Chrome, Microsoft Edge, Mozilla Firefox, Safari, and Opera.

Numerous templates and examples are available to accelerate web development, although it is also possible to build projects from scratch using Bootstrap's grid system, utility classes, and prebuilt components. The framework simplifies front-end development because developers can reuse ready-made elements and customize them according to the needs of their websites. In addition, Bootstrap provides responsive design features that automatically adapt content to different screen sizes, making it suitable for desktop computers, tablets, and mobile devices.

BOOTSTRAP (IV)

Bootstrap is a front-end framework that provides a collection of **CSS classes** and **JavaScript components** for building responsive websites and web applications.

To link a website's code with the file that contains the style rules, bootstrap.min.css, the procedure is the same as with any other CSS stylesheet.

Developers can include Bootstrap in their projects either by downloading the framework files or by linking to a **Content Delivery Network (CDN)**. Once included, Bootstrap styles and components can be applied through predefined CSS classes, significantly reducing development time and ensuring a consistent user interface.

BOOTSTRAP (V)

2. **Include Bootstrap's CSS and JS.** Place the `<link>` tag in the `<head>` for our CSS, and the `<script>` tag for our JavaScript bundle (including Popper for positioning dropdowns, popovers, and tooltips) before the closing `</body>`. Learn more about our CDN links.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" re
  </head>
  <body>
    <h1>Hello, world!</h1>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.
  </body>
</html>
```

BOOTSTRAP (VI)

Bootstrap provides a wide variety of predefined styles and components that help developers create responsive and visually consistent websites.

The framework includes CSS files that define the appearance of elements such as forms, tables, buttons, and navigation menus, as well as JavaScript files that provide interactive components such as modals, alerts, dropdown menus, carousels, and other dynamic interface elements.

Once the Bootstrap stylesheet has been linked to the HTML document, the styles can be applied simply by using Bootstrap's predefined classes within the HTML elements. Developers can either use these styles as provided or customize them to meet the specific requirements of a project.

BOOTSTRAP (VII)

Once Bootstrap is included in the project, its predefined classes can be applied directly to HTML elements to create responsive and consistent user interfaces.

```
project/
│
├── css/
│   └── bootstrap.min.css
│
├── js/
│   └── bootstrap.bundle.min.js
│
└── index.html
```

CONTAINER

Bootstrap provides a set of tools and essential components for the development of websites and web applications. The Bootstrap layout system is one of the foundations on which responsive web designs are built.

To control the layout and structure of a web page, Bootstrap provides several container types. The most commonly used are **.container**, **.container-fluid** and the responsive containers such as **.container-sm**, **.container-md**, **.container-lg**, **.container-xl** and **.container-xxl**, which adapt to different screen sizes.

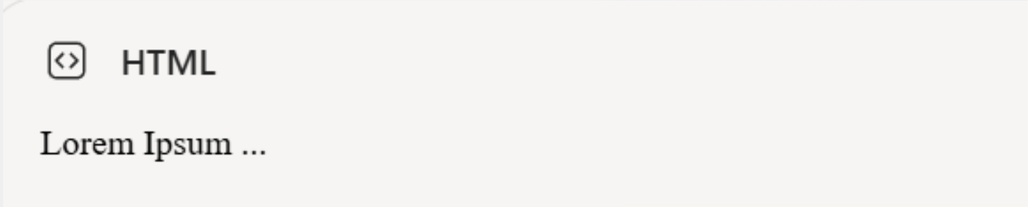
The **.container** class is the most common container element in Bootstrap. It creates a centered layout with a maximum width that changes according to the viewport size.

The **.container-fluid** class spans the entire width of the viewport, regardless of screen size.

CONTAINER (II)

The following example shows a .container-fluid element containing a centered .container element, inside which a paragraph of text is placed.

```
<div class="container-fluid bg-primary">
    <div class="container">
        <p>Lorem Ipsum ...</p>
    </div>
</div>
```



As shown in the resulting output, the paragraph inside the .container element is constrained by the margins and maximum width defined for that container type, while the .container-fluid element occupies the full width of the browser window.

¿ANY QUESTION?